```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; /** * @title NISTTrack_Compliance * @dev On-chain AI Agent Safety & Compliance Enforcer aligned with NIST AI RMF & SP 800-53 Overlays. */ contract NISTTrack_Compliance { struct AgentAuditRecord { string agentId; uint256 coherenceScoreK; // Scaled by 10000 (e.g., 9500 = 0.95) uint256 timestamp; bytes32 promptHash; bool isCompliant; } address public immutable founder; address public nistEscrowVault; uint256 public constant MIN_COHERENCE_THRESHOLD = 9500; // 0.95 K-Score floor mapping(address => AgentAuditRecord) public agentRegistry; mapping(address => bool) public authorizedAuditors; event AuditLogged(address indexed agentAddress, string agentId, uint256 kScore, bool compliant); event ComplianceViolation(address indexed agentAddress, string agentId, uint256 kScore); modifier onlyAuditor() { require(authorizedAuditors[msg.sender] || msg.sender == founder, "Access Denied: Not authorized"); _; } constructor(address _nistEscrowVault) { founder = msg.sender; nistEscrowVault = _nistEscrowVault; authorizedAuditors[msg.sender] = true; } function setAuditor(address _auditor, bool _status) external { require(msg.sender == founder, "Only founder"); authorizedAuditors[_auditor] = _status; } /** * @dev Submits a periodic cryptographic telemetry audit for an AI agent. */ function recordAudit( address _agentAddress, string calldata _agentId, uint256 _kScore, bytes32 _promptHash ) external onlyAuditor { bool compliant = (_kScore >= MIN_COHERENCE_THRESHOLD); agentRegistry[_agentAddress] = AgentAuditRecord({ agentId: _agentId, coherenceScoreK: _kScore, timestamp: block.timestamp, promptHash: _promptHash, isCompliant: compliant }); emit AuditLogged(_agentAddress, _agentId, _kScore, compliant); if (!compliant) { emit ComplianceViolation(_agentAddress, _agentId, _kScore); // Trigger automatic circuit breaker / lock } } function verifyCompliance(address _agentAddress) external view returns (bool) { return agentRegistry[_agentAddress].isCompliant; } }
```